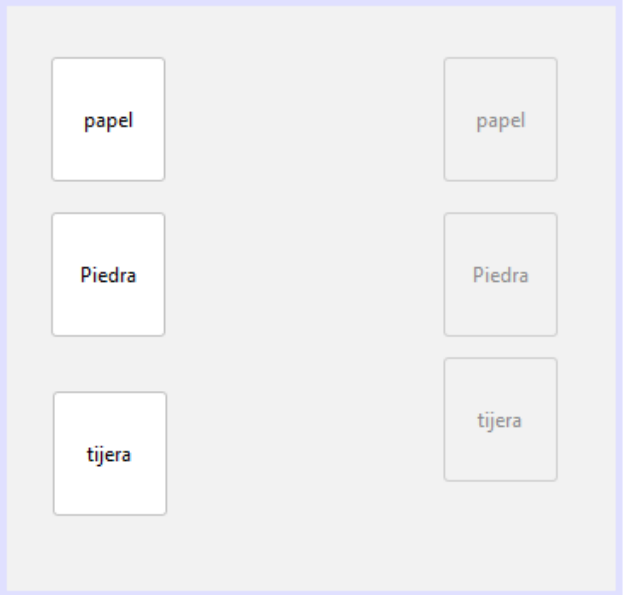
Pasos a seguir para realizar la actividad:

1. Crear un proyecto nuevo, en mi caso lo llamo “NewClass1”.
2. Creamos un JFrame, el cual lo llamo PPT.
3. En el JFrame hacemos el diseño de la app como se muestra en la actividad.



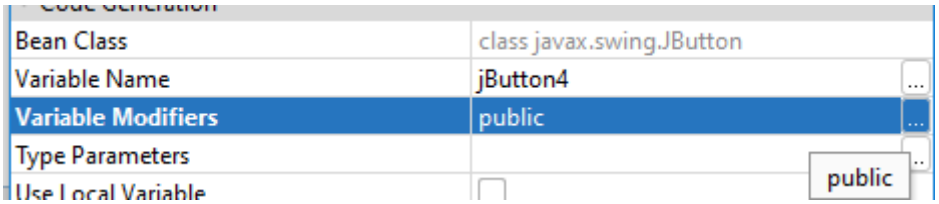
La lógica de la aplicación la haremos en la clase principal NewClass1.

Tenemos dos clases, NewClass1 y PPT, Si yo desde NewClass1 quiero acceder a los atributos o componentes que se encuentran en la clase PPT, deberé hacer una instancia de la clase PPT en NewClass1 y poner los atributos o componentes como públicos.

Primero crearé un array de botones y guardaré los tres botones que se encuentran en el JFrame, los que debe de seleccionar la máquina de forma automática (los botones de la derecha)



Me creo una instancia de la clase PPT, la cual estoy nombrando como nf.
Para poder acceder a los botones, primero tengo que acceder a nf, ya que es la clase que contiene los botones. Para ello debo hacer los botones públicos, haciendo **click derecho sobre el componente botón -> properties -> code**, y cambiamos el valor de variable Modifiers por públic.



De esta forma ya puedo acceder a los componentes o atributos de una clase desde otra.

En la NewClass1, creamos un método nuevo que será el que llamaremos en los botones con los que nosotros interactuamos.

```
public  static void BtnRm()
{
Random rm = new Random(); //instancia de la clase random
 int NumRandom = rm.nextInt(bound: allBtn.length); //genera un número aleatoria entre 0 y 3
 allBtn[NumRandom].setBackground(bg: Color.red); //accede al array de botones con el NumRandom
                                                 //y cambia el color del botón a rojo

}
```

Ejecutamos esté método que pertenece a la NewClass1, en el evento del botón que se encuentra en la clase PPT. Para ello el método debe ser público y estático.

```
private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:

NewClass1.BtnRm();
```

Ahora mismo al pulsa el primer botón me seleccionará en rojo uno de la derecha de forma aleatoria, si vuelvo a pulsar de nuevo, se me volverá a pintar en rojo otro botón sin reestablecerse en blanco el anterior. Para ello, lo que podemos hacer antes de pintar en rojo el botón, es un for y pintar en blanco todos los botones que tenemos en el array

```
public  static void BtnRm()
{
Random rm = new Random(); //instancia de la clase random
 int NumRandom = rm.nextInt(bound: allBtn.length); //genera un número aleatoria entre 0 y 3

 for (int i = 0; i < allBtn.length; i++) {
        allBtn[i].setBackground(bg: Color.white);
    }

 allBtn[NumRandom].setBackground(bg: Color.red); //accede al array de botones con el NumRandom
                                                 //y cambia el color del botón a rojo



}
```

El método que estoy llamando en el botón es el mismo que llamo en el resto de botones. Pero el programa debe saber si desde donde llamo el método es en el botón tijera, papel o piedra. Para eso añado un parámetro de entrada a la función que especifique el nombre del botón desde donde se ejecuta el método.

```
public  static void BtnRm(String nombreBoton)
{
Random rm = new Random(); //Instancia de la clase random
 int NumRandom = rm.nextInt(bound: allBtn.length); //genera un número aleatorio entre 0 y 3

 for (int i = 0; i < allBtn.length; i++) {
        allBtn[i].setBackground(bg: Color.white);
    }

 allBtn[NumRandom].setBackground(bg: Color.red); //accede al array de botones con el NumRandom
                                                 //y cambia el color del botón a rojo
```

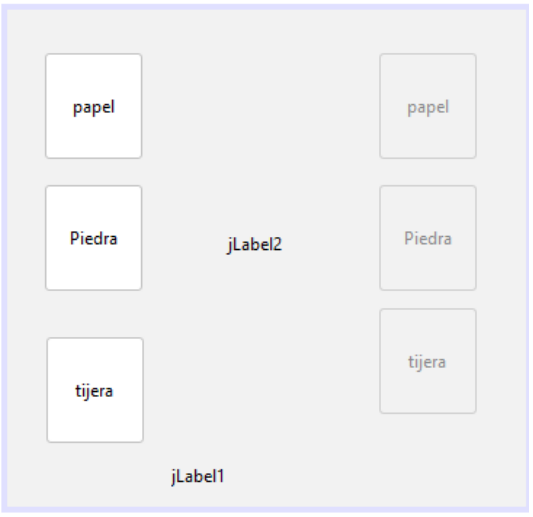
De esta manera cuando ahora llame el método en el evento del botón, deberé pasarle su nombre a través del jButton1.getText();

```
    private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {
        // TODO add your handling code here:

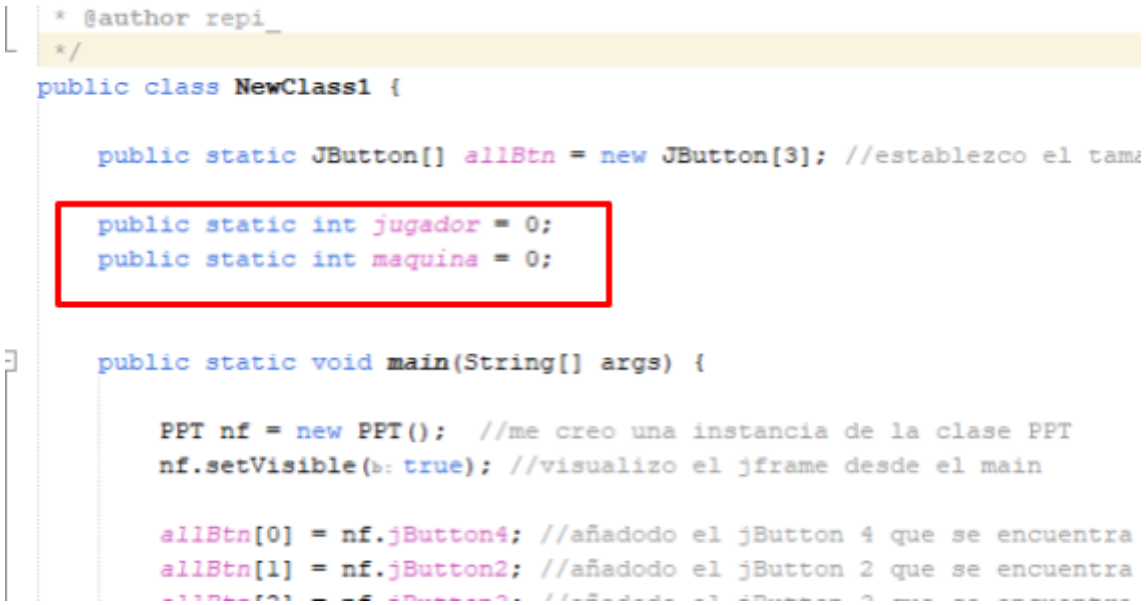
    NewClass1.BtnRm(nombreBoton:jButton1.getText());
```

De esta forma ya podré crear las condiciones en el método BtnRm comparándo el botón que yo he seleccionado con el que me ha marcado la máquina de forma aleatoria para establecer el sistema de puntos.

Creamos un jLabel2 en el jframe para mostrar el sistema de puntos y otro jLabel1 para que nos muestre en pantalla el texto, pierdes, ganas o empate.



Creamos dos variables de tipo int que me almacene la puntuación en la clase principal



En el método BtnRm, creamos las condiciones con los distintos casos de empate, perder y ganar, comparando el nombre del botón que selecciona el jugador con el nombre del botón que selecciona la máquina de forma aleatoria. Si el usuario selecciona el botón con el nombre piedra y la máquina selecciona el botón con el nombre piedra, tendremos empate.

```
public  static void BtnRm(JButton boton)
{
Random rm = new Random(); //instancia de la clase random
int NumRandom = rm.nextInt(bound: allBtn.length); //genera un número aleatoria entre 0 y 3

 for (int i = 0; i < allBtn.length; i++) {
        allBtn[i].setBackground(bg: Color.white);
    }

 allBtn[NumRandom].setBackground(bg: Color.red); //accede al array de botones con el NumRandom
                                                 //y cambia el color del botón a rojo
 boton.setBackground(bg: Color.white);

 if(boton.getText().equals(anObject: allBtn[NumRandom].getText()))
 {

    boton.setBackground(bg: Color.orange);
    allBtn[NumRandom].setBackground(bg: Color.orange);
    PPT.jLabel1.setText(text: "Empate");
 }

 else if(boton.getText().equals(anObject: "papel") && allBtn[NumRandom].getText().equals(anObject: "tijera"))
 {
  PPT.jLabel1.setText(text: "pierde");
  maquina++;
  PPT.jLabel2.setText(jugador + "     "+ maquina);
 }

 else if(boton.getText().equals(anObject: "papel") && allBtn[NumRandom].getText().equals(anObject: "Piedra"))
 {
  PPT.jLabel1.setText(text: "gano");
  jugador++;
  PPT.jLabel2.setText(jugador + "     "+ maquina);
 }
```

```
<build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-jar-plugin</artifactId>
        <version>3.1.0</version>
        <configuration>
          <archive>
            <manifest>
              <mainClass>${exec.mainClass}</mainClass>
            </manifest>
          </archive>
        </configuration>
      </plugin>
    </plugins>
  </build>
```